# END-TO-END MACHINE LEARNING PROJECT PLAN

A Python-based campaign response prediction system

Project domain: Marketing analytics, Binary classification

# Document Control

| Item | Details |
|---|---|
| Document title | End-to-End Machine Learning Project Plan |
| Version | 1.0 |
| Prepared on | 27 October 2026 |
| Project | Python-based Campaign Response Prediction |
| Document status | Proposed plan; implementation and model results are not yet claimed |
| Planning effort | 30–35 hours to research, design, document and review the plan |
| Delivery horizon | Six weeks after project approval; dates can be adjusted to the academic calendar |

# Table of Contents


1. Executive Summary
2. Introduction and Background
3. Research and Ideation
4. Problem Definition
5. Objectives, Scope and Success Criteria
6. Data Strategy and Governance
7. End-to-End ML Pipeline Design
8. Model Development and Evaluation Plan
9. Deployment and Monitoring Strategy
10. Project Timeline, Milestones and Deliverables
11. Critical Path and Dependencies
12. Resource Requirements
13. Risk Management
14. Quality Assurance and Acceptance Criteria
15. Strategic Roadmap and Future Enhancements
16. Estimated 30–35 Hour Planning Work Breakdown
17. Conclusion
18. References

# 1. Executive Summary

This project proposes an end-to-end machine learning (ML) workflow in Python to estimate whether a customer contacted during a marketing campaign is likely to subscribe to a bank term deposit. The intended users are marketing analysts and campaign managers who need a consistent way to prioritize follow-up activity and compare likely campaign responses.

The project will use the publicly available UCI Bank Marketing dataset as a reproducible learning dataset. It contains customer and campaign attributes and a binary outcome indicating whether a client subscribed. The full bank-marketing dataset is documented as containing 45,211 instances and 16 features; a separate enriched version is also available. The exact file and schema will be recorded during data acquisition.

The proposed workflow includes business framing, data sourcing, exploratory analysis, preprocessing, baseline development, model comparison, evaluation, explainability, packaging and a small demonstration interface. Logistic Regression will provide an interpretable baseline, while Decision Tree and Random Forest models will be evaluated as non-linear alternatives. The final model will be selected based on validation performance, business usefulness, calibration and operational simplicity not accuracy alone.

This is a planning document. Any metrics, model scores or business benefits must be measured during implementation and must not be presented as achieved results before testing.

# 2. Introduction and Background

## 2.1 Context

Machine learning can help organizations estimate patterns from historical data and support decisions where outcomes are uncertain. In marketing, a response-prediction model can estimate the probability that a customer will respond to an offer. A campaign team may then use these scores to test prioritization strategies, while retaining human oversight and respecting customer privacy.

## 2.2 Why this project domain?

- A clear target exists: whether the customer subscribed to a term deposit (yes/no).
- The data includes numerical and categorical attributes, so it exercises common preprocessing tasks.
- The dataset is public and documented, allowing the project to be reproduced without requesting private customer records.
- The domain offers meaningful evaluation challenges, including class imbalance, decision thresholds, and possible leakage from features observed only after a call.

## 2.3 Proposed project output

The planned deliverable is a documented, reproducible Python prototype that accepts eligible campaign/customer attributes and returns a response probability plus a threshold-based label. It will include a model evaluation report and a basic demonstration interface. It is not intended to make automatic decisions about access to financial services or to replace campaign staff.

# 3. Research and Ideation

## 3.1 Brief literature and resource review

The UCI Machine Learning Repository is a public source for benchmark datasets used in ML education and research. Its Bank Marketing dataset describes direct marketing calls made by a Portuguese banking institution, with the prediction target indicating whether a client subscribed to a term deposit. The dataset is suitable for a structured classification exercise because the target and feature descriptions are documented.

Moro, Rita and Cortez published a study using data-driven methods to predict the success of bank telemarketing. Their work provides a domain-specific precedent for analyzing marketing campaign response and is cited by the UCI dataset record. The project will use the dataset as a teaching benchmark, not as evidence that the same model performance will transfer to a modern campaign.

The scikit-learn documentation describes standard model selection and evaluation tools, including train/test splitting, cross-validation, classification metrics, preprocessing pipelines and model persistence. These will guide the implementation so preprocessing is fitted only on training data and the final test set remains isolated until the model-selection process is complete.

### 3.2 Ideation and selection rationale

| Candidate problem | Advantages | Reason for decision |
|---|---|---|
| Campaign response prediction | Business-relevant binary target; mixed data types; clear evaluation metrics | Selected: practical and feasible for a student-scale Python pipeline |
| House-price estimation | Clear regression task and widely available datasets | Good alternative, but less connected to campaign planning |
| Image classification | Visually engaging; supports deep learning | Requires image processing and potentially more compute than needed for this plan |

### 3.3 Research questions

1. Can historical customer and campaign features estimate subscription likelihood better than a simple majority-class baseline?
2. Which model offers a useful balance between recall of likely responders, precision, probability quality and interpretability?
3. How do class imbalance, feature leakage and threshold choice affect the usefulness of the results?
4. Can the trained pipeline be saved, loaded and used consistently through a small prototype interface?

## 4. Problem Definition

### 4.1 Problem statement

Marketing teams may spend time contacting customers who are unlikely to accept an offer, while potentially missing customers who are more receptive. The project will investigate whether historical campaign data can be used to estimate the probability of a positive response and provide decision support for campaign prioritization.

### 4.2 Machine learning task

| Element | Definition |
|---|---|
| Task type | Supervised binary classification |
| Prediction unit | One customer/campaign contact record |
| Target label | Subscription outcome: yes or no (dataset target commonly named y) |
| Inputs | Available customer profile and prior/current campaign attributes documented in the selected dataset |
| Output | Estimated probability of a positive response and a label based on a chosen threshold |
| Primary users | Marketing analysts, campaign managers and project evaluators |
| Intended use | Analysis and prioritization support; no automatic customer treatment |

### 4.3 Input features and data availability

The final feature list will be confirmed against the exact selected dataset version. Candidate attributes in the standard bank-full file include age, job, marital status, education, credit default status, housing loan, personal loan, contact channel, month, day of month, campaign contact count, days since a previous contact, previous contacts and previous

campaign outcome. The project will document each field, its data type, allowable values and missing/unknown-value convention.

The ‘duration’ field needs special treatment: it describes the duration of the current call and is only known after the call has ended. Including it in a model intended to prioritize customers before a call would introduce target leakage. It will therefore be excluded from the primary pre-contact model; a separate benchmark may be considered only if its limitations are explicitly explained.

### 4.4 Challenges and assumptions

- The positive response class is expected to be smaller than the negative class; accuracy alone may hide poor detection of responders.
- Some categorical values may be recorded as 'unknown'. These values will be examined rather than silently treated as ordinary missing values.
- Historical campaign conditions may differ from current conditions, limiting generalization.
- Associations learned by a model do not prove that a feature causes a customer to respond.
- The public data is historical and from one institutional context; it is not representative of all customers or banks.

### 4.5 Expected outcomes

- A reproducible dataset and preprocessing workflow.
- A baseline and at least two candidate classifiers compared on the same held-out data.
- A model evaluation report with metrics, confusion matrix and threshold discussion.
- A saved model pipeline and a prototype prediction interface, subject to successful testing.
- A limitations, privacy, fairness and monitoring note.

## 5. Objectives, Scope and Success Criteria

### 5.1 Objectives

5. Define a measurable business question and binary prediction target.
6. Acquire and document an appropriate public dataset and its licence/attribution requirements.
7. Profile data quality, target balance, feature types and potential leakage.
8. Build a preprocessing pipeline that handles categorical and numerical variables consistently.
9. Compare a simple baseline with Logistic Regression, Decision Tree and Random Forest candidates.
10. Evaluate performance using class-sensitive metrics and a held-out test set.
11. Explain model limitations and identify the factors that most influence predictions where supported by the selected model.
12. Package the selected model and demonstrate inference through a simple local interface.
13. Create technical documentation, experiment records and a future monitoring plan.

### 5.2 Scope included

- Structured tabular data, binary classification, exploratory analysis and classical ML.
- Offline training and evaluation, with a local demonstration interface such as Streamlit.
- Reproducibility through fixed random seeds, dependency versions and saved artifacts.

### 5.3 Out of scope

- Production integration with a real bank's customer relationship management (CRM) system.
- Live use of private customer data, automated customer contact or automated eligibility decisions.
- Guaranteed business savings or guaranteed predictive performance.
- Deep learning, complex cloud infrastructure and continuous retraining in the first version.

### 5.4 Proposed success criteria

| Area | Proposed acceptance criterion |
|---|---|
| Data validation | Dataset source, version, target, schema and known limitations are documented; no unexplained duplicate or invalid-value issue remains. |
| Reproducibility | A clean environment can run the documented pipeline from raw data to evaluation artifacts. |
| Leakage control | Pre-contact prediction excludes features that are only available after the target decision/event. |
| Model comparison | At least three candidate approaches, including a dummy baseline, are compared using a consistent split and metrics. |
| Evaluation | Report includes precision, recall, F1, ROC-AUC, PR-AUC where available, confusion matrix and class balance. |
| Model selection | Final choice is justified by validation results and use-case trade-offs; test set is used only for final reporting. |
| Prototype | A saved pipeline loads successfully and handles valid and invalid inputs with clear messages. |
| Documentation | Setup instructions, assumptions, limitations, dataset citation and risk register are complete. |

# 6. Data Strategy and Governance

## 6.1 Data source and acquisition

Primary source: UCI Bank Marketing dataset, dataset record 222. The team will download the selected file from the official UCI page or use the documented loader. The exact file (for example, bank-full.csv versus bank-additional-full.csv), retrieval date, row count, column list, target definition, licence and citation will be recorded before analysis.

## 6.2 Data quality checklist

- Confirm the file opens and matches the documented schema.
- Check row and column counts, data types, target labels and duplicate rows.
- Inspect missing values and special tokens such as 'unknown'; document treatment.
- Check impossible or implausible numeric values and category spelling consistency.
- Plot the target distribution and key feature distributions.
- Check whether records are time-ordered and whether a chronological split is more realistic than a random split.
- Record transformations and keep an untouched copy of the original source file.

## 6.3 Data split strategy

A proposed starting split is 70% training, 15% validation and 15% final test, using stratification where appropriate. If the selected file's ordering represents time and the intended deployment predicts future campaigns, the team will consider a chronological split instead of a purely random split. Split strategy must reflect the intended use and be documented. The test set must not be used to tune hyperparameters or choose the decision threshold.

## 6.4 Privacy, security and ethics

- Use the public dataset for the prototype; do not upload private customer records to unapproved services.
- Avoid collecting direct identifiers. Restrict access to project files and remove unnecessary copies.
- Use predictions as decision support, not as a basis for denying services or treating customers unfairly.
- Inspect performance across relevant groups where appropriate and lawful; report limitations rather than claiming fairness from one metric.
- Document data provenance, licence and citation, and do not imply that the historical dataset is current.
- Protect saved model files and logs from accidental publication if they contain sensitive information.

# 7. End-to-End ML Pipeline Design

| Phase | Planned activities | Main output |
|---|---|---|
| 1. Business understanding | Define decision point, user, target, costs of false positives/negatives and success criteria | Problem brief and evaluation plan |
| 2. Data acquisition | Download selected public dataset; record source, version, licence and schema | Versioned raw data and data card |
| 3. Data understanding | Descriptive statistics, target balance, missingness, distributions and leakage review | EDA notebook and data-quality report |
| 4. Data preparation | Train/validation/test split; imputation if needed; one-hot encoding; scaling where model requires it | Reusable preprocessing pipeline |
| 5. Baseline modelling | Majority-class dummy classifier; initial Logistic Regression | Baseline metrics |
| 6. Candidate modelling | Decision Tree and Random Forest; optional class weights and limited tuning | Experiment comparison table |
| 7. Evaluation | Validation metrics, confusion matrix, ROC/PR curves, calibration and threshold analysis | Model selection report |
| 8. Explainability | Coefficients or feature importance; verify interpretations are not causal claims | Interpretation and limitations note |
| 9. Packaging | Save fitted preprocessing-plus-model pipeline; record package versions | Model artifact and environment file |
| 10. Prototype deployment | Local Streamlit app or command-line prediction script; input validation and error handling | Runnable demo |
| 11. Monitoring plan | Define drift indicators, performance review cadence and retraining triggers | Monitoring checklist and roadmap |

## 7.1 Proposed Python project structure

```
campaign_response_ml/
  data/              # raw/processed data (do not commit restricted data)
  notebooks/         # exploration and experiments
  src/               # reusable preprocessing, training and inference code
  models/            # versioned model artifacts
  reports/           # metrics, plots and final report
  app.py             # local demonstration interface
  requirements.txt   # pinned dependencies
  README.md          # setup and reproducibility instructions
  .gitignore         # excludes environments, secrets and large/private files
```

## 7.2 Preprocessing design

- Use a scikit-learn Pipeline and ColumnTransformer to keep transformations consistent between training and inference.
- For numerical columns, apply a documented imputation strategy if needed; scale for Logistic Regression if useful.
- For categorical columns, use one-hot encoding with safe handling of previously unseen categories.
- Fit every preprocessing operation on the training partition only, then apply the fitted transformation to validation and test data.
- Keep the target separate from input features and explicitly remove leakage-prone columns.

## 7.3 Reproducibility

- Set and record random seeds for splits and stochastic models.
- Record Python and library versions, dataset version and command used to run each experiment.

- Save configuration, selected features, evaluation metrics and model artifact together.
- Use version control for code and documentation; avoid committing secrets, credentials or unnecessary data copies.

# 8. Model Development and Evaluation Plan

## 8.1 Candidate models

| Model | Role | Strengths | Risks / considerations |
|---|---|---|---|
| DummyClassifier | Required baseline | Shows performance from a simple reference strategy | Not intended to learn useful patterns |
| Logistic Regression | Interpretable baseline | Fast, probabilistic output; coefficients can be inspected | May miss non-linear relationships; scaling and encoding need care |
| Decision Tree | Non-linear candidate | Easy to visualize; captures interactions | Can overfit without depth/leaf constraints |
| Random Forest | Primary ensemble candidate | Captures non-linear effects and interactions; generally robust | Less transparent than a single tree; requires tuning and careful explanation |

## 8.2 Evaluation metrics

- Precision: of records predicted positive, the proportion that are truly positive.
- Recall: of truly positive records, the proportion identified by the model.
- F1-score: harmonic mean of precision and recall; useful when both matter.
- ROC-AUC: measures ranking across thresholds, but can look optimistic when positives are rare.
- PR-AUC / average precision: focuses on precision-recall trade-offs for the positive class.
- Confusion matrix: counts true positives, false positives, true negatives and false negatives.
- Calibration: checks whether predicted probabilities align with observed response rates.

The team will not report accuracy in isolation. The model threshold will be selected using validation data and an explicit discussion of the relative cost of missed responders versus unnecessary follow-up. The final test report will describe performance at the selected threshold and, where useful, compare it with a simple operational baseline.

## 8.3 Experiment protocol

14. Create a baseline with a dummy classifier and record the positive-class prevalence.
15. Build preprocessing and Logistic Regression as a first complete pipeline.
16. Train Decision Tree and Random Forest candidates using the same training data and evaluation protocol.
17. Tune a small, predeclared set of hyperparameters with cross-validation on training data; do not tune against the test set.
18. Compare validation results and choose a model based on metrics, calibration, simplicity and the use-case trade-off.
19. Freeze the model and threshold, evaluate once on the held-out test set, and record all final results.
20. Inspect errors and limitations; do not claim causal explanations from feature importance.

## 8.4 Error analysis

Review false positives and false negatives, examine whether errors cluster in particular feature ranges, and check whether performance differs materially across relevant subgroups. Avoid drawing conclusions from very small subgroup samples. If the model is poorly calibrated or does not outperform the baseline on useful metrics, report that result honestly and revise the approach rather than forcing a deployment.

# 9. Deployment and Monitoring Strategy

## 9.1 Prototype deployment

The initial deployment target is a local demonstration application. Streamlit is a suitable optional interface for a student project because it can expose input fields and display the estimated response probability without building a separate front end. A command-line script is an acceptable fallback if time is limited.

- Load the exact saved preprocessing-and-model pipeline, not a separately reconstructed transformation.
- Validate required fields, ranges and category choices; show actionable error messages.
- Display probability, threshold-based class and a short note explaining that the output is an estimate.
- Do not expose training data or unnecessary personal details in the interface.
- Keep the model artifact and dependency versions aligned; test the app in a clean environment.

## 9.2 Production readiness considerations

A public or enterprise deployment would require additional security, privacy, legal, access-control, logging, monitoring, incident-response and change-management review. Those activities are outside the first prototype and should be treated as a separate approval gate.

## 9.3 Monitoring plan

| Signal | Review approach | Possible action |
|---|---|---|
| Input data drift | Compare feature distributions with training reference | Investigate source changes; assess need to retrain |
| Prediction drift | Track rate of positive predictions over time | Check for data pipeline changes or campaign shifts |
| Outcome performance | When labels become available, calculate precision, recall, PR-AUC and calibration | Review threshold and model quality |
| Data quality | Monitor missingness, unexpected categories and invalid values | Pause predictions or route for correction |
| Operational reliability | Record load failures, latency and version used | Rollback to a known stable artifact if necessary |

For the student prototype, monitoring can be demonstrated using a checklist and sample report. No real customer data should be collected merely to make the monitoring section appear complete.

# 10. Project Timeline, Milestones and Deliverables

The implementation schedule below is a proposed six-week plan. Week 1 begins after approval; actual calendar dates can be assigned by the project team. The separate planning effort estimate appears in Section 16.

| Phase / week | Tasks | Milestone / deliverable | Owner (suggested) |
|---|---|---|---|
| Week 1 — Define | Confirm user and prediction point; review sources; finalize target, scope and metrics | M1: Approved project charter and dataset choice | Project lead + data analyst |
| Week 2 — Acquire & inspect | Download data; verify schema/licence; EDA; quality and leakage review | M2: Data card, EDA report and split strategy | Data analyst |
| Week 3 — Prepare & baseline | Implement split and preprocessing; train dummy and Logistic Regression baseline | M3: Reproducible baseline pipeline | ML developer |
| Week 4 — Candidate models | Train Decision Tree and Random Forest; limited CV tuning; error analysis | M4: Validation comparison and model recommendation | ML developer + reviewer |
| Week 5 — Final evaluation & prototype | Freeze model; test once; save artifact; build local interface and validate inputs | M5: Evaluation report and working demo | ML developer + QA |
| Week 6 — Documentation & | Run clean-environment test; | M6: Final submission package | Whole team |

| | | | |
|---|---|---|---|
| review | complete README, limitations, risk and future roadmap; present | and review sign-off | |

### 10.1 Milestone exit criteria

| Milestone | Exit criteria |
|---|---|
| M1 — Charter approved | Problem, target, scope, stakeholders and initial metrics agreed. |
| M2 — Data understood | Source and licence recorded; data quality, class balance, leakage and split strategy documented. |
| M3 — Baseline complete | Pipeline executes end to end and baseline metrics are saved. |
| M4 — Candidate comparison | Models compared on validation data; selection rationale and trade-offs documented. |
| M5 — Prototype ready | Held-out evaluation completed; artifact loads; interface validates inputs. |
| M6 — Final review | Documentation complete, clean setup tested, limitations acknowledged and deliverables archived. |

### 10.2 Deliverable register

| ID | Deliverable | Format / evidence |
|---|---|---|
| D1 | Project charter and requirements | DOCX / project notes |
| D2 | Dataset card and EDA | Markdown or PDF report, notebook and plots |
| D3 | Preprocessing and modelling pipeline | Python source code |
| D4 | Experiment log and evaluation report | CSV/JSON metrics, plots and written analysis |
| D5 | Selected model artifact | Joblib or equivalent saved pipeline |
| D6 | Prototype interface | Streamlit app or command-line inference tool |
| D7 | Reproducibility package | README, requirements file, version notes and .gitignore |
| D8 | Risk, monitoring and future roadmap | Sections in final report |

## 11. Critical Path and Dependencies

The critical path is the sequence of tasks that determines the earliest feasible completion date. The most important dependency chain is: problem and target definition → dataset/version confirmation → leakage and split decisions → preprocessing pipeline → baseline and candidate models → validation-based selection → final test evaluation → packaging → clean-environment testing → final sign-off.

| Dependency | Why it matters | Control / contingency |
|---|---|---|
| Target definition before modelling | The target determines features, split and metrics | Approve a short data dictionary before writing training code |
| Dataset and feature availability before preprocessing | Schema or version changes can invalidate transformations | Pin the chosen dataset version and validate the schema automatically |
| Leakage review before model comparison | Leaky features can make performance look unrealistically high | Review each feature against the prediction time; exclude post-outcome features |
| Split strategy before tuning | Repeatedly changing the split undermines fair comparison | Record split method and random seed; use a time split if justified |
| Model selection before final test | Using test results to tune creates optimistic evaluation | Keep the test partition untouched until selection is frozen |
| Saved pipeline before app integration | Inference must apply the same preprocessing as training | Persist one complete pipeline and test known sample inputs |
| Stable app before documentation freeze | Interface changes may alter setup steps | Reserve Week 6 for clean install, smoke tests and corrections |

### 11.1 Parallel work opportunities

- Literature review and initial environment setup can proceed while the dataset is being selected.

- Documentation templates, risk register and interface wireframe can be prepared while baseline experiments run.
- EDA charts and data dictionary can be developed in parallel once the schema is confirmed.
- Final documentation should not freeze until evaluation and prototype behavior are stable.

### 11.2 Schedule buffer

Reserve approximately 10–15% of available implementation time for unexpected data issues, package conflicts, slow experiments or report revisions. If delayed, prioritize a reproducible baseline, a fair evaluation and complete documentation before optional visual polish or advanced models.

## 12. Resource Requirements

### 12.1 Human resources

| Role | Main responsibilities | Estimated involvement |
|---|---|---|
| Project lead / coordinator | Scope, schedule, decisions, review and final submission | 2–4 hours per week |
| Data analyst | Dataset validation, EDA, feature dictionary and data quality | 4–6 hours concentrated in Weeks 1–2 |
| ML developer | Preprocessing, training, tuning, evaluation and packaging | 6–10 hours concentrated in Weeks 3–5 |
| QA / reviewer | Reproducibility, edge cases, leakage review and documentation checks | 2–4 hours across Weeks 4–6 |

In a solo academic project, one student may perform all roles. The role table is a responsibility checklist, not a requirement to recruit a team.

### 12.2 Software and libraries

| Tool / library | Purpose | Need |
|---|---|---|
| Python 3.11 or a compatible supported version | Core language and environment | Required |
| Jupyter Notebook / VS Code | Exploration, editing and execution | One preferred environment |
| pandas, NumPy | Tabular data handling and numerical operations | Required |
| Matplotlib / Seaborn | Charts and exploratory visualizations | Recommended |
| scikit-learn | Splitting, preprocessing, models, metrics and pipelines | Required |
| joblib | Persist the fitted pipeline | Recommended |
| Streamlit | Optional local demonstration interface | Optional |
| Git / GitHub | Version control and review | Recommended; do not publish sensitive data |
| pytest | Automated checks for preprocessing and inference | Recommended |

### 12.3 Hardware and budget

- A typical laptop with 8 GB RAM or more should be adequate for this tabular dataset; 16 GB provides more headroom.
- A GPU is not expected to be necessary for the proposed classical ML models.
- A stable internet connection is needed to obtain the dataset and install packages.
- The project can be completed using free/open-source software; optional hosting may introduce costs or account requirements.

### 12.4 Environment setup

Create a virtual environment, install pinned package versions and record them in requirements.txt. A proposed starter set is pandas, numpy, scikit-learn, matplotlib, seaborn, joblib and optionally streamlit and pytest. Exact versions should be pinned after a successful clean run, rather than copied without testing.

# 13. Risk Management

| Risk | Likelihood / impact | Mitigation | Contingency / owner |
|---|---|---|---|
| Target leakage from post-call fields | Medium / High | Define prediction moment; exclude duration from pre-contact model; document every feature | Retrain without leakage-prone feature; ML developer |
| Class imbalance causes misleading accuracy | High / High | Use precision, recall, F1, PR-AUC, confusion matrix and threshold analysis | Compare class weighting or threshold alternatives; ML developer |
| Historical data not representative of current use | High / Medium | State dataset provenance and limits; avoid claims of current transferability | Treat as academic prototype only; project lead |
| Overfitting during tuning | Medium / High | Use cross-validation on training data; reserve a final test set | Simplify model and repeat with correct split; ML developer |
| Schema or category mismatch at inference | Medium / High | Pipeline-based preprocessing; validate input schema and unseen categories | Return clear error and block invalid predictions; developer/QA |
| Environment or package incompatibility | Medium / Medium | Pin dependencies and test in a clean environment | Use a known stable environment and document versions; QA |
| Privacy or accidental data exposure | Low / High | Use public data, avoid identifiers, review repository contents and access | Remove exposed artifacts and follow institutional reporting process; project lead |
| Unfair or harmful use of scores | Medium / High | Limit intended use to decision support; assess subgroup performance; retain human oversight | Do not deploy if evaluation raises unresolved concerns; reviewer |
| Schedule slippage | Medium / Medium | Weekly checkpoints, critical path and 10–15% buffer | Drop optional features before core validation/documentation; coordinator |
| Unclear business success measure | Medium / Medium | Agree on false-positive/false-negative trade-offs and threshold rationale | Report several threshold options instead of claiming one universal optimum; project lead |

### 13.1 Risk review cadence

Review risks at the end of each week and whenever the dataset, target, feature set or deployment scope changes. Record each risk's status, owner, next action and any decision made. High-impact risks must be addressed before the model is demonstrated as ready.

# 14. Quality Assurance and Acceptance Criteria

### 14.1 Testing plan

| Test level | Example checks | Pass condition |
|---|---|---|
| Data tests | Expected columns, valid target labels, data types, duplicates and null counts | Failures are resolved or explicitly documented |
| Pipeline tests | Train/test isolation, transformer fit behavior, unknown categories | No test leakage; transformations run consistently |
| Model tests | Baseline runs, candidate metrics saved, deterministic seed where possible | Experiments are repeatable and comparable |
| Inference tests | Valid input, missing required field, invalid numeric range and unseen category | Valid inputs return output; invalid inputs show safe messages |
| Artifact tests | Save then reload pipeline and compare | Reloaded pipeline behaves consistently |

| | | |
|---|---|---|
| | predictions | |
| Documentation tests | Fresh setup instructions and dataset citation | A reviewer can reproduce the core workflow |

### 14.2 Final acceptance checklist

- ☐ Problem statement, scope, target and intended use are explicit.
- ☐ Dataset source, exact file/version, licence and attribution are included.
- ☐ Leakage review and split strategy are documented.
- ☐ Preprocessing is fitted only on training data and is reused at inference.
- ☐ Dummy baseline and candidate models are evaluated consistently.
- ☐ Class-sensitive metrics and a confusion matrix are reported.
- ☐ Model choice and threshold are justified with validation data.
- ☐ Final test results are clearly separated from validation results.
- ☐ Known limitations, fairness considerations and privacy risks are acknowledged.
- ☐ Saved pipeline and prototype are tested in a clean environment.
- ☐ README, requirements file, project structure and final report are complete.

## 15. Strategic Roadmap and Future Enhancements

| Stage | Focus | Possible work | Gate to proceed |
|---|---|---|---|
| Stage 1 — Academic prototype (Weeks 1–6) | Reliable baseline and documented pipeline | Complete this plan, compare models, evaluate and demo locally | Core acceptance checklist passed |
| Stage 2 — Robustness (next iteration) | Improve reliability and interpretation | Calibration, threshold analysis, temporal validation, more error analysis | Results stable across appropriate validation scenarios |
| Stage 3 — Responsible use review | Assess governance and practical value | Subgroup checks, stakeholder review, data protection and documented use policy | Risks reviewed and approved by relevant stakeholders |
| Stage 4 — Controlled pilot (future only) | Test with authorized data and oversight | Secure integration, access controls, monitoring and rollback | Formal organizational approval, privacy/security review and success metrics |
| Stage 5 — Maintenance | Detect degradation and manage change | Scheduled performance reviews, drift checks, retraining and model versioning | Evidence that ongoing value outweighs maintenance cost |

### 15.1 Future enhancements

- Compare a time-aware validation approach if campaign chronology is central to deployment.
- Test probability calibration and decision thresholds against realistic follow-up capacity.
- Add a simple model card that explains intended use, data limitations, metrics and prohibited uses.
- Automate data-quality checks and smoke tests as part of a lightweight continuous integration workflow.
- Only consider more complex models if they show a meaningful, reproducible benefit over simpler baselines.
- If authorized current data ever becomes available, reassess data governance, drift and external validity before any pilot.

## 16. Estimated 30–35 Hour Planning Work Breakdown

The assignment specifies approximately 30–35 hours for the planning task. The table below allocates 33 hours to research, project design and document preparation. This is distinct from the six-week implementation schedule.

| Planning activity | Hours | Evidence produced |
|---|---|---|
| Research ML lifecycle and identify candidate domains | 4 | Short literature review and source notes |
| Select problem and define target, inputs, | 3 | Problem definition and scope |

| | | |
|---|---|---|
| users and boundaries | | |
| Review dataset, schema, licence and data-quality concerns | 4 | Data strategy and feature/leakage notes |
| Design end-to-end pipeline and model comparison plan | 5 | Pipeline design, evaluation protocol and experiment plan |
| Build timeline, milestones, dependencies and critical path | 4 | Schedule and milestone register |
| Analyze resources, environment and computational needs | 3 | Resource plan |
| Create risk register and mitigation strategies | 3 | Risk management table |
| Plan deployment, testing, monitoring and roadmap | 3 | Deployment and future roadmap |
| Format, reference, proofread and validate the document | 4 | Final DOCX and submission checklist |
| Total | 33 | Complete project plan |

### 16.1 Planning controls

- Use the same project name, target and dataset version throughout the document.
- Keep assumptions separate from verified dataset facts and measured results.
- Use a weekly review to check completeness, not just page count.
- Reserve the final review session for citation checks, formatting, table readability and consistency.

## 17. Conclusion

This plan defines a feasible route from business framing to a reproducible Python ML prototype for marketing campaign response prediction. It prioritizes data quality, leakage prevention, a fair evaluation protocol, appropriate metrics for an imbalanced target, clear risk ownership and a realistic staged delivery schedule. The public dataset makes the project practical for academic work, while its historical context and feature limitations require careful interpretation.

The project should be considered successful not simply because a model is trained, but because the process is reproducible, the results are honestly evaluated, the model's limits are understood and the prototype can be tested safely. Any move from a student demonstration to real operational use would require additional validation, privacy/security review and explicit organizational approval.

## 18. References


21. [1] Moro, S., Rita, P., & Cortez, P. (2014). Bank Marketing [Dataset]. UCI Machine Learning Repository. Dataset 222. https://doi.org/10.24432/C5K306. Official dataset page: https://archive.ics.uci.edu/dataset/222/bank+marketing
22. [2] scikit-learn developers. scikit-learn User Guide: Model selection, preprocessing, pipelines and model evaluation. https://scikit-learn.org/stable/user_guide.html
23. [3] scikit-learn developers. Model evaluation: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html
24. [4] scikit-learn developers. Pipeline and composite estimators. https://scikit-learn.org/stable/modules/compose.html
25. [5] IBM. CRISP-DM (Cross-Industry Standard Process for Data Mining) overview and methodology resources. https://www.ibm.com/docs/en/spss-modeler